

LLM Driven VFX: Automating Script Generation and Model Validation with LLMs

Aditya Roy Chowdhury

adityaro@usc.edu

Abstract

Visual Effects (VFX) workflows rely heavily on manual scripting in tools like Autodesk Maya, creating barriers for smaller teams and independent artists due to the steep learning curve. While recent advances such as Stable Diffusion and Runway Gen-2 demonstrate the potential of natural language-driven content creation, they are limited to 2D outputs and do not integrate with professional 3D pipelines. To address this gap, we propose an LLM-driven assistant that translates natural language prompts into executable MEL scripts, renders 3D scenes in Maya, and validates outputs using vision models such as CLIP. Our approach fine-tunes Code LLaMA-Instruct 7B using LoRA on a curated MEL dataset and incorporates a visual validation framework combining geometric and semantic checks. This hybrid system automates asset creation, ensures output fidelity, and makes advanced VFX tools more accessible to a broader range of creators.

1. Introduction

Visual Effects (VFX) are a cornerstone of modern media creation across film, television, and gaming. Professional tools like Autodesk Maya, Houdini, and Blender allow artists to create complex digital environments, with Maya particularly favored for its robust scripting capabilities through Python and MEL. However, the steep learning curve associated with manual scripting creates barriers for smaller studios and independent creators, limiting access to advanced VFX production workflows.

Our project addresses this gap by proposing an LLM-driven VFX assistant that automates the translation of natural language prompts into executable MEL scripts, renders the corresponding scenes in Maya, and validates the outputs using vision models. The research objective is to reduce manual effort in asset creation, democratize access to professional-grade VFX tools, and streamline production pipelines through AI assistance.

We have refined our technical approach by selecting

Code LLaMA-Instruct 7B as the foundation for script generation, and applying Low-Rank Adaptation (LoRA) for efficient fine-tuning on a custom-curated dataset of MEL scripting tasks. To ensure the geometric and semantic fidelity of generated outputs, we incorporated a visual validation framework based on CLIP encoders (ViT B32 Transformer) with contrastive loss as the objective which is minimised.

These adjustments expanded the original scope from simple script generation toward a full hybrid pipeline that combines natural language processing, 3D rendering, and vision-based validation. This approach not only improves script accuracy but also ensures that the visual and structural outputs meet professional production standards.

By specializing large language models for domain-specific VFX tasks and integrating them with computer vision-based validation, our project contributes a novel tool to the broader research field of robotic perception and AI-driven content creation. It advances the state of the art by demonstrating how LLMs and vision models can work in tandem to automate complex, multi-stage creative workflows, bridging the gap between language, code, and 3D asset generation.

2. Related Work

2.1. The Evolution of AI in VFX

The integration of Artificial Intelligence (AI) into Visual Effects (VFX) has gained momentum, complementing traditional 3D modeling, animation, and rendering processes. While software like Autodesk Maya, Houdini, and Blender have long been industry standards, the growing complexity of digital content has necessitated automation and AI-assisted tools [1]. AI-driven advancements in generative modeling and procedural workflows have begun reshaping content creation, but their impact on real-world VFX pipelines remains limited.

2.2. Advancements in AI for Content Creation

AI has significantly impacted creative automation, with breakthroughs in text-to-image (Stable Diffusion), text-to-

video (Runway Gen-2), and procedural 3D model generation (Shap-E, Point-E) [10]. These technologies have demonstrated AI’s capability to generate highquality digital assets, but they primarily function as standalone tools rather than being embedded into existing VFX software. Some of these involves generating 3D models using textual information [6], or converting a 2D image of an object to a 3D CAD model with VLM [14]. Both the aforementioned methods utilizes the power of LLM’s such as BERT or GPT-4V to capture the semantic meaning of instructions given by the designer. The gap between AI-powered asset generation and fully integrated AI-driven VFX workflows highlights the need for specialized solutions [3]. AI-enhanced VFX solutions have primarily focused on procedural modeling, texture synthesis, and automated animation techniques. Machine learning has been leveraged for motion tracking and GAN-based rendering, such as NVIDIA’s AIassisted denoising and upscaling tools [5]. However, these implementations require extensive datasets and high computational power, often making them impractical for independent artists and smaller studios [7]. An AI-driven approach tailored for direct integration with

2.3. The Need for AI-Driven Assistance in Maya

Although AI has been widely adopted in other creative fields, there is no dedicated AI system designed to enhance Maya’s procedural workflows and scripting environment. Existing research suggests that AI can streamline procedural modeling and automate script generation, yet there is no finetuned model designed for Maya’s Python/MEL scripting ecosystem [2]. By bridging this gap, a specialized AI-powered assistant could optimize repetitive tasks, improve workflow efficiency, and enhance accessibility for artists working within Maya [8].

3. Methodology

The project pipeline has been systematically designed, specifying the model used at each stage, the script type used to generate the final output (e.g., 3D object file) and the evaluation metrics selected for validation of the final output against ground truth images.

3.1. Training Dataset Construction

Given the absence of any publicly available datasets that conform to our desired input-output structure, namely, from a natural language prompt to a MEL script and then to a rendered 3D asset, we constructed a custom dataset in a different format for finetuning which consists of input and response pairs.

- **Input (Natural Language Prompt):** A concise queries of general information about Maya. For example, “What’s the MEL command to create a degree 3 curve on surface”

- **Output/Response:** The output contains the answer to the input which consists of a mixture of MEL scripts, python scripts, and natural language.

Due to the lack of renderable MEL scripts on the internet, we elected to finetune the model with simply general information as well so the model can learn some MEL specific information. To construct this dataset, we adopted a two-step methodology. First, we collected a diverse set of responses from publicly accessible sources such as Autodesk’s official Maya documentation. Next, we employed gemini-2.0-flash to generate natural language prompts that closely reflect the intent and structure of each response.

3.2. Model Fine-Tuning

Firstly, Code LLaMA Instruct [11] with 7 billion parameters is trained to generate MEL scripts from natural language descriptions. The model is fine-tuned using LoRA (Low-Rank Adaptation) to efficiently adapt a pre-trained LLM to the MEL scripting domain [4]. LoRA introduces trainable low-rank matrices, which reduces finetuning time significantly. The following are

- Prompts are tokenized from natural language descriptions.
- Supervised fine-tuning is applied using MEL scripts as targets.
- Cross-entropy loss is used to optimize output accuracy.

This process enables the model to learn MEL syntax and prompt semantics, producing executable scripts suitable for direct use in Maya.

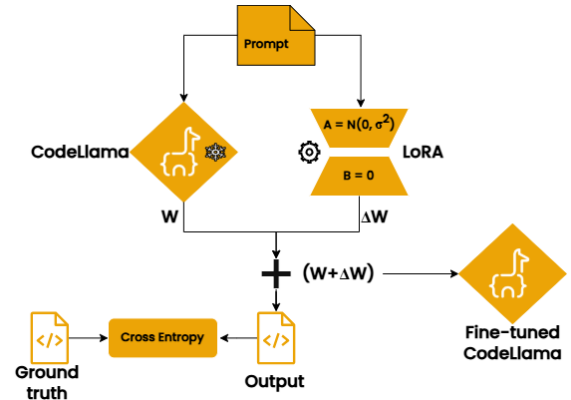


Figure 1. Fine-tuning CodeLlama with LoRA

3.3. Validation Framework

After generating and rendering the MEL script, we validate the output 3D model to ensure alignment to the input prompt based on following factors.

Visual Similarity Visual similarity assesses how closely a generated 3D render matches the ground-truth image, em-

phasizing human perception of shape and structure. It captures perceptual alignment, structural features like contours and part layout, texture or shading cues if present, and view-point consistency—ensuring renders appear similar from matching angles.

- **DINOv2-Based Similarity Validation**

We use DINOv2, a self-supervised Vision Transformer, to assess visual similarity by comparing cosine similarity between embeddings of the generated and ground-truth renders. DINOv2 captures high-level shape and layout features, making it suitable for validating whether the generated object perceptually aligns with the prompt.

Geometric Similarity Geometric similarity evaluates how closely the 3D shape and structure of a generated object match the ground-truth object. It focuses on point-wise surface alignment (e.g., the distance between corresponding points on the generated and reference objects), overall shape conformity, and spatial consistency measured through volume and surface area comparisons. Below are metrics used to measure geometric similarity.

- **Chamfer Distance:** Chamfer Distance [13] measures geometric similarity between two point clouds P and Q by computing the average squared distance from each point in one set to its nearest neighbor in the other:

$$\frac{1}{|P|} \sum_i \|p_i - \text{NN}_Q(p_i)\|^2 + \frac{1}{|Q|} \sum_j \|q_j - \text{NN}_P(q_j)\|^2$$

where $\text{NN}_R(s_i)$ is the point in R that is closest to s_i . It quantifies the overall shape alignment between the generated and ground-truth 3D objects.

- **Volume and Surface Area Comparison:** These metrics assess size and shape conformity. Volume comparison measures consistent over- or under-scaling, while surface area reveals deviations in detail or smoothness, helping identify noisy or oversimplified geometry.
- **Hausdorff Distance:** Measures the largest point-wise deviation between two point clouds, capturing worst-case shape errors [12]. Useful for identifying structural mismatches not reflected in average-based metrics like Chamfer Distance.

Semantic Similarity Semantic similarity measures how well the meaning of the prompt matches the visual content of the generated output.

- **VIT-Based Comparison using CLIP embedding:** CLIP embedding is able to embed both language and image into the same embedding space [9]. To measure semantic alignment between the generated image and the input prompt, we use CLIP with the ViT-B/32 transformer backbone. As shown in Figure 2, CLIP encodes both the

text prompt and the rendered image into vector embeddings, and computes cosine similarity between them. A higher CLIP score indicates stronger semantic correspondence. This allows us to validate whether the generated output semantically represents the intended meaning of the prompt.

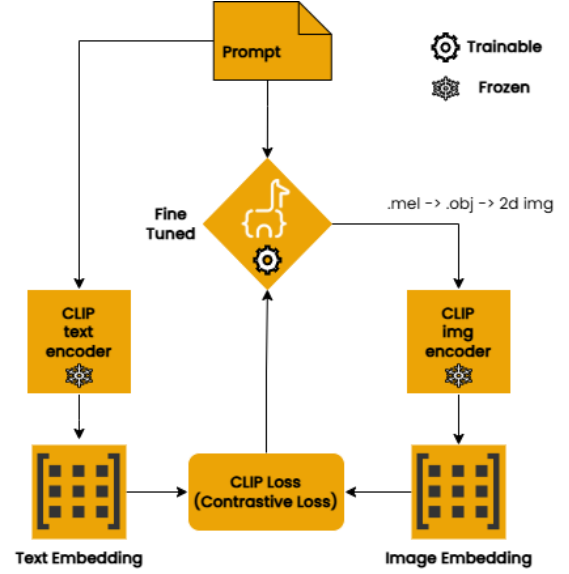


Figure 2. Training with Contrastive Loss

Contrastive Validation Loop To enhance alignment between generated scripts and their intended prompts, we introduced a contrastive loss using CLIP embeddings as shown in Figure 2. The text and image encoders from CLIP were frozen, and the model was retrained to minimize cosine similarity between the embeddings of the prompt and the corresponding rendered output. This contrastive loss guided the model to generate MEL scripts that more accurately reflected the semantic content of the input prompt, reinforcing it as a useful validation tool within the generation pipeline.

4. Results

4.1. Dataset

The analysis of the dataset intended for fine-tuning Code Llama for Maya scripting revealed several insights significant to our project objectives. The dataset comprises three types of data: MEL scripts, Python scripts, and general informational content derived from Maya documentation.

Figure 3 illustrates the prompt length distribution measured in tokens. The dataset primarily consists of short prompts, with most prompts under 50 tokens, indicating concise requests or instructions. There is a sharp peak at

fewer than 25 tokens, suggesting that typical prompts are succinct, focusing on specific functional queries rather than complex or narrative-style instructions. The presence of very few lengthy prompts exceeding 150 tokens highlights potential outliers or complex scenarios.

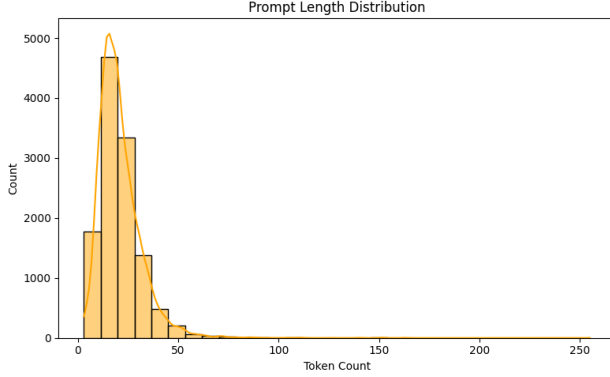


Figure 3. Prompt Length Distribution

The response type analysis reveals a significant predominance of general information responses (approximately 45,000 instances), compared to Python and MEL responses. Python responses constitute a smaller, yet substantial portion (around 15,000 instances), while MEL responses are comparatively minimal. This distribution aligns with the nature of the Maya documentation, emphasizing descriptive general information over script-specific examples.

Model Type	Valid(%)	Invalid(%)
Code LLaMA-Instruct-7B	13	87
Gemini	60	40
Finetuned Code LLaMA	92	8

Table 1. Syntax validity on test set

Function	Count
getFaceVertexBlindDataIndex	146,232
range	107,331
relative	97,826
create	86,556
provided	86,187

Table 2. Top Functions in Scripts

Explainability	Count
Explained	43,833
Code Only	19,079

Table 3. Explainability of Code Snippets

Analysis of frequently occurring functions (Table 2) pro-

vides insights into the essential commands and functions most relevant for Maya scripting. The most common function was `getFaceVertexBlindDataIndex`, with 146,232 occurrences, followed by widely used Python functions such as `range`, `relative`, and `create`. This underscores the need to prioritize these commonly occurring functions during the fine-tuning process to ensure the model captures fundamental Maya functionalities effectively.

Table 3 summarizes the explainability of responses, categorizing them into explained or code-only entries. The data shows that explanations accompany most of the code snippets (43,833 explained entries), substantially outnumbering the code-only snippets (19,079 entries). This is beneficial as explained examples aid in the model’s comprehension and context understanding, potentially enhancing its capability to generate more accurate and contextually appropriate responses.

A slight imbalance is observed in the dataset, primarily characterized by the significantly higher volume of general informational responses compared to Python and MEL scripts. This imbalance might affect the model’s scripting-specific capabilities and warrants consideration during training to ensure balanced performance across various response types.

We manually generated 100 test prompt/response pairs, where each prompt is a natural language description of a 3D scene. Prompts range from simple cases involving single objects (e.g., cube, sphere, pyramid) to more complex scenarios with spatial relationships, such as “a pyramid sitting on top of a cube” or “a torus beside a cone.”

We evaluated our fine-tuned model against two baselines: the original (non-finetuned) Code LLaMA model and Gemini. These comparisons help isolate the impact of domain-specific fine-tuning and contrastive validation.

4.2. Syntax Validity

From Table 1, we observe that out of 100 test prompts, the baseline Code LLaMA-Instruct-7B model was able to generate only 13% MEL scripts with valid syntax. Gemini performed better with 60% valid scripts, while our fine-tuned model generated 92% syntactically correct outputs. This indicates that even though our fine-tuning was performed on general MEL information, the model effectively learned core syntactic patterns from the Maya documentation, enabling reliable script generation.

4.3. ViT-Based Visual Similarity Evaluation

As shown in Figure 4, the fine-tuned model achieves consistently higher cosine similarity with ground-truth renders than Gemini, indicating stronger visual alignment. This confirms that fine-tuning improves the model’s ability to generate scenes that better match the prompt’s intended structure and layout.

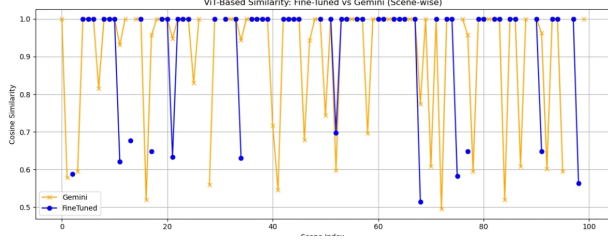


Figure 4. Scene-wise cosine similarity between generated and ground-truth renders using DINOv2.

4.4. Chamfer Distance Analysis

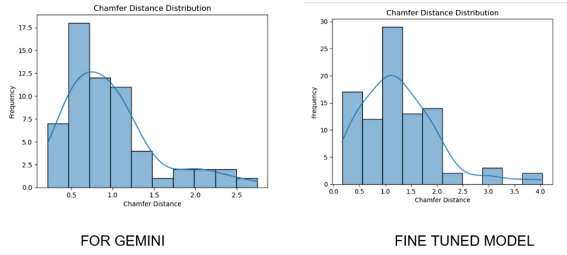


Figure 5. Chamfer Distance Distribution

Chamfer Distance captures surface alignment by averaging nearest-point distances between point clouds of generated and ground-truth meshes. As shown in Figure 5, Gemini tends to produce tighter alignments under 1.0, while our fine-tuned model shows more outputs in the 1.0–2.0 range—still indicating good geometric fidelity with slightly more variation.

4.5. Volume Comparison

Figure 6 shows that our fine-tuned model more closely tracks the volume of ground-truth objects, reflecting better shape complexity and geometric fidelity. In contrast, Gemini frequently underestimates volume, producing flatter outputs.

4.6. Surface Area Comparison

Surface area reflects object detail and smoothness. As shown in Figure 7, our fine-tuned model more closely tracks ground-truth values, preserving complexity. In contrast, Gemini is less consistent, often misestimating details and reducing geometric accuracy.

4.7. Hausdorff Distance

Figure 8 shows that our fine-tuned model consistently maintains lower Hausdorff distances relative to ground truth, indicating better worst-case structural accuracy. Gemini shows larger fluctuations, highlighting more frequent shape deviations from the ground truth.

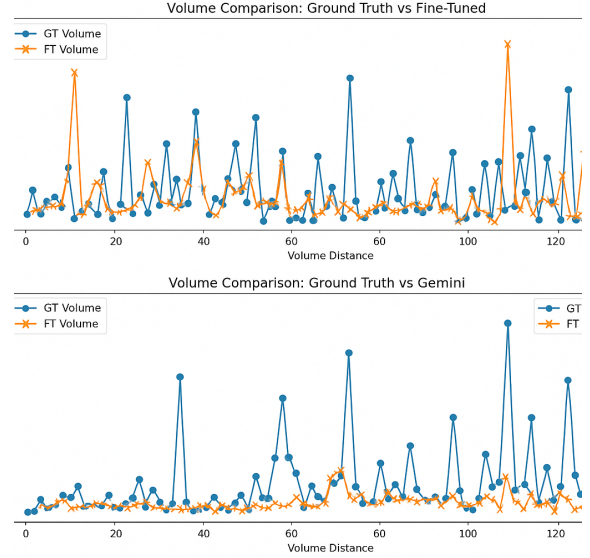


Figure 6. Volume comparison between ground truth, fine-tuned model, and Gemini.

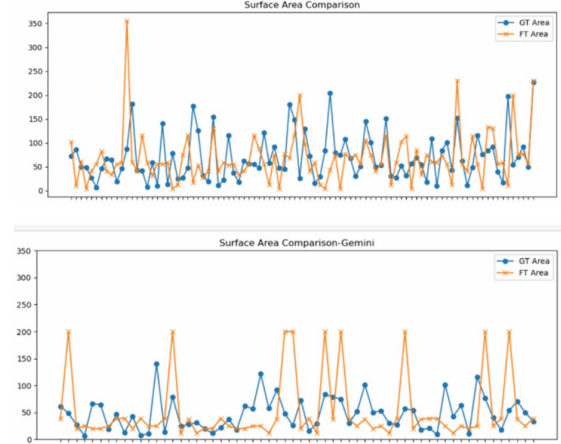


Figure 7. Surface area comparison between ground truth, fine-tuned model (top), and Gemini (bottom).

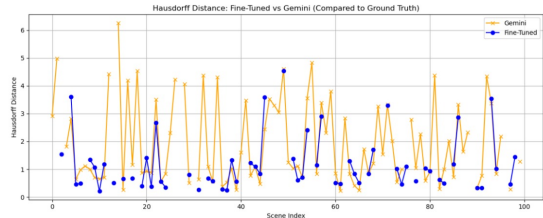


Figure 8. Hausdorff Distance comparison: Fine-Tuned vs Gemini (relative to ground truth).

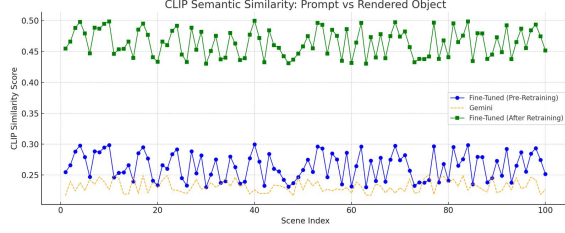


Figure 9. CLIP similarity comparison between prompt and rendered object. Retraining refers to additional training to maximize the cosine similarity as specified in section 4.

4.8. Clip Similarity

Figure 9 shows that the fine-tuned model consistently outperforms Gemini in CLIP similarity, indicating better semantic alignment between prompts and generated objects. Scores improved further after retraining the model using contrastive loss on frozen CLIP embeddings, guiding the system to produce more meaningful and prompt-consistent outputs.

Across all evaluation metrics—geometric alignment, semantic consistency, and visual similarity—our fine-tuned model consistently outperforms the Gemini baseline.

4.9. Prompt Guided Object Detection

Dataset. For initial evaluation, we constructed a small dataset comprising 30 synthetic `.obj` files (15 cubes, 15 spheres), each rendered from five viewpoints to produce a total of 150 images.

Classifier Results. The ViT classifier was trained to distinguish cubes and spheres.

Table 4. Object Classification Accuracy

Class	Precision	Recall
Cube	0.96	0.95
Sphere	0.97	0.96

The classifier achieved a test accuracy of **0.96**, indicating robust performance in distinguishing object classes based on rendered geometry.

Similarity Model Results. The Siamese ViT model was evaluated on 2,000 pairwise comparisons of `.obj` renders.

Table 5. Average Similarity Scores

Pair Type	Average Score
Cube vs Cube	0.91
Sphere vs Sphere	0.90
Cube vs Sphere	0.34

These results show clear separation and reliability of the similarity metric for detecting geometric mismatches.

Prompt Evaluation Examples. Figure 10 shows rendered outputs with predictions compared against their respective natural language prompts.

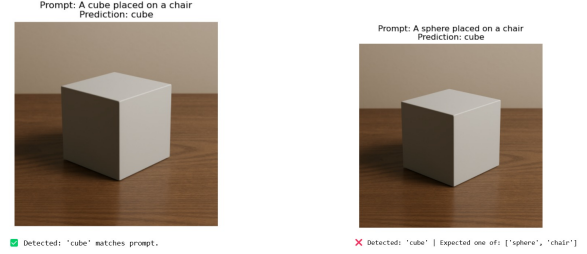


Figure 10. Prompt: “A cube placed on a chair” (left) vs. “A sphere placed on a chair” (right). In the first case, the classifier correctly detects a cube and matches the prompt. In the second case, the object detection module detects a cube while the prompt expects a sphere, correctly flagging a semantic mismatch.

These examples demonstrate that prompt-guided object detection can effectively verify the semantic correctness of LLM-generated assets by aligning object predictions with natural language descriptions.

4.10. Summary of Results

To consolidate the evaluation across various metrics, we present a summary comparing the performance of our fine-tuned Code LLaMA model against the Gemini baseline. This overview highlights improvements in syntax validity, geometric fidelity, and semantic alignment achieved through fine-tuning and validation.

Table 6. Overall Performance Comparison between Fine-Tuned Code LLaMA and Gemini

Metric	Fine-Tuned Code LLaMA	Gemini Baseline	Comments
Syntax Validity	92%	60%	Fine-tuning dramatically improved script correctness.
Chamfer Distance	1.0–2.0 range (good)	Mostly <1.0 (slightly tighter)	Fine-tuned model shows good surface alignment.
Volume Accuracy	Closely matches ground truth	Frequently underestimates	Fine-tuned preserves true object volume better.
Surface Area Accuracy	Consistent with ground truth	More variation, less consistent	Fine-tuned captures surface details better.
Hausdorff Distance	Lower deviation from ground truth	Higher worst-case deviations	Fine-tuned maintains better structure.
CLIP Semantic Similarity	Higher alignment to prompts	Lower alignment	Fine-tuned captures intended meaning better.
Prompt-Guided Object Detection Accuracy	96%	(Baseline not evaluated)	Robust object classification and verification.

5. Conclusion

This project introduced an LLM-driven VFX assistant that automates MEL script generation from natural language prompts and validates 3D outputs using vision models. Fine-tuning Code LLaMA-Instruct 7B with LoRA on a curated MEL dataset significantly improved script validity and semantic alignment compared to baseline models. The visual validation framework, combining Chamfer Distance,

surface comparisons, and CLIP-based semantic similarity, effectively ensured geometric and visual fidelity.

Our system advances the integration of language models and computer vision within VFX workflows, contributing to the broader field of robotic perception by demonstrating AI-driven automation in multi-stage creative tasks. Practically, it reduces manual scripting overhead and increases accessibility for smaller studios and independent artists.

A key limitation was the limited diversity of renderable MEL scripts in the training data, restricting scene complexity. Future work will focus on expanding the dataset, incorporating Reinforcement Learning with Human Feedback (RLHF), and extending validation to dynamic scenes and animations to enhance robustness and scalability.

References

- [1] Autodesk. *Maya Documentation*. Autodesk, 2022. 1
- [2] X. Chai et al. Ai-assisted animation: Automating rigging and motion capture processing. *Journal of Computer Graphics*, 38(2):67–82, 2021. 2
- [3] T. Gao et al. Machine learning for procedural content generation in film and gaming. *Computational Creativity Journal*, 29(1):55–72, 2023. 2
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 2
- [5] R. Huang et al. Neural networks in procedural modeling: Opportunities and challenges. *Graphics and AI Journal*, 19(4):98–113, 2021. 2
- [6] Mohammad Sadil Khan, Sankalp Sinha, Sheikh Talha Uddin, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. 2
- [7] Y. Kim et al. Gan-based rendering and its application in vfx pipelines. *Journal of Artificial Intelligence in Media*, 17(2):78–92, 2022. 2
- [8] H. Liang et al. Intelligent asset creation: The future of ai in visual effects. *Journal of Computer Vision and VFX*, 40(3):134–152, 2023. 2
- [9] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763. PMLR, 2021. 3
- [10] A. Ramesh et al. Text-to-image generation with diffusion models. In *NeurIPS Conference Proceedings*, 2022. 2
- [11] Baptiste Roziere and et al. Code llama: Open foundation models for code. *Meta AI*, 2023. 2
- [12] William J. Rucklidge. Efficiently locating objects using the hausdorff distance. *International Journal of Computer Vision*, 24(3):251–270, 1997. 3
- [13] Tong Wu, Liang Pan, Junzhe Zhang, Tai Wang, Ziwei Liu, and Dahua Lin. Density-aware chamfer distance as a comprehensive metric for point cloud completion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. Code available at https://github.com/wutong16/Density_aware_Chamfer_Distance. 3
- [14] Yang You, Mikaela Angelina Uy, Jiaqi Han, Rahul Thomas, Haotong Zhang, Suya You, and Leonidas Guibas. Img2cad: Reverse engineering 3d cad models from images through vlm-assisted conditional factorization, 2024. 2